

```

DROP SUBSCRIPTION
db_destination=# truncate t1;
TRUNCATE TABLE
db_destination=# CREATE SUBSCRIPTION sub_all_tables CONNECTION
'user=replicarole password=LearnPostgreSQL host=pg_pub port=5432
dbname=db_source' PUBLICATION all_tables_pub;
NOTICE:  created replication slot "sub_all_tables" on publisher
CREATE SUBSCRIPTION

```

Now, if we check both servers, the primary server and the replica server will have all data aligned. On the pg_pub server, we have the following:

```

db_source=# select * from t1;
 id | name
----+-----
  1 | Linux
  2 | FreeBSD
  4 | Minix
  3 | Windows
  5 | Unix
(5 rows)

```

On the replica pg_sub server, we have this:

```

db_destination=# select * from t1;
 id | name
----+-----
  1 | Linux
  2 | FreeBSD
  4 | Minix
  3 | Windows
  5 | Unix
(5 rows)

```

Now, if we execute the query again on the pg_stat_replication, our logical replication will be found:

```

db_source=# \x
Expanded display is on.
db_source=# select * from pg_stat_replication;

```

```

-[ RECORD 1 ]-----+-----
pid          | 337
usesysid     | 16477
username     | replicarole
application_name | sub_all_tables
client_addr  | 192.168.144.2
client_hostname |
client_port  | 57826
backend_start | 2023-06-16 15:24:36.905319+00
backend_xmin  |
state        | streaming
sent_lsn     | 0/1DD0BC0
write_lsn    | 0/1DD0BC0
flush_lsn    | 0/1DD0BC0
replay_lsn   | 0/1DD0BC0
write_lag    |
flush_lag    |
replay_lag   |
sync_priority | 0
sync_state   | async
reply_time   | 2023-06-16 15:27:47.24564+00

```

DDL commands

In the previous section, we said that logical replication does not replicate DDL commands, but what happens if we apply a DDL statement on a primary server that is already replicated using logical replication? The DDL commands are as follows:

- CREATE
- ALTER
- DROP
- RENAME
- TRUNCATE
- COMMENT

Suppose now we want to add a field on the t1 table of the primary server, pg_pub:

```

db_source=# alter table t1 add description varchar(64);
ALTER TABLE

```

Let's now try to make a DML command on the `pg_pub` server. Some examples of DML commands follow:

- INSERT
- DELETE
- UPDATE

For example, say we tried to delete a record from the `t1` table of the `pg_pub` server:

```
db_source=# delete from t1 where id=5;
DELETE 1
```

On the `pg_pub` server, we would have the following:

```
db_source=# select * from t1;
 id | name  | description
----+-----+-----
  1 | Linux |
  2 | FreeBSD |
  4 | Minix |
  3 | Windows |
(4 rows)
```

On the `pg_sub` server, though, we would still have this:

```
db_destination=# select * from t1;
 id | name
----+-----
  1 | Linux
  2 | FreeBSD
  4 | Minix
  3 | Windows
  5 | Unix
(5 rows)
```

If we examine `postgresql.log` on the `pg_sub` server, we'll see this:

```
learn_postgresql_sub_1 | 2023-06-16 15:30:57.973 UTC [364] ERROR:
logical replication target relation "public.t1" is missing replicated
column: "description"
```

```
learn_postgresql_sub_1 | 2023-06-16 15:30:57.973 UTC [364] CONTEXT:
processing remote data for replication origin "pg_16480" during message
type "DELETE" in transaction 783, finished at 0/1DD5380
learn_postgresql_sub_1 | 2023-06-16 15:30:57.975 UTC [1] LOG:  background
worker "logical replication worker" (PID 364) exited with exit code 1
^CERROR: Aborting.
```

The logical replication does not work anymore because the logical replication target relation `public.t1` is missing some replicated columns, as the server log reported. If we want to solve this problem, we must execute the DDL on the replica server:

```
db_destination=# alter table t1 add description varchar(64);
ALTER TABLE
```

Now, if we check the records on the `pg_sub` server, we have the same records that are present on the `pg_pub` server:

```
db_destination=# select * from t1;
 id | name   | description
----+-----+-----
  1 | Linux  |
  2 | FreeBSD |
  4 | Minix  |
  3 | Windows |
(4 rows)
```



DDL commands must always be replicated on the replica servers.

Disabling logical replication

In the previous section, we used the `DROP SUBSCRIPTION` command to drop a subscription. There may be cases where we cannot use this command directly. For example, suppose that the primary server becomes unreachable and we need to drop the subscription on the replica server. If we try to execute a `DROP SUBSCRIPTION` command, we will get the following response:

```
db_destination=# drop subscription sub_all_tables ;
ERROR:  could not connect to publisher when attempting to [..]
```

```
HINT: Use ALTER SUBSCRIPTION ... SET (slot_name = NONE) to disassociate
the subscription from the slot.
```

PostgreSQL suggests using `ALTER SUBSCRIPTION ... SET (slot_name = NONE)` to disassociate the subscription from the slot. The problem is that we cannot execute this command before having disabled the subscription. In fact, if we try to perform the command suggested by PostgreSQL now, we will get this:

```
db_destination=# alter subscription sub_all_tables SET (slot_name =
NONE);
ERROR: cannot set slot_name = NONE for enabled subscription
```

The correct steps that we have to execute are as follows:

1. Disable the subscription.
2. Set `slot_name` to `NONE`.
3. Drop the subscription.

We have to perform the following three statements:

```
db_destination=# alter subscription sub_all_tables disable;
ALTER SUBSCRIPTION
db_destination=# alter subscription sub_all_tables SET (slot_name = NONE);
ALTER SUBSCRIPTION
db_destination=# drop subscription sub_all_tables ;
DROP SUBSCRIPTION
```

These are the correct steps if we want to drop a subscription when the primary server becomes unreachable. We can also use the `ALTER SUBSCRIPTION sub_name DISABLE` command to detach the subscription from the publication, and the `ALTER SUBSCRIPTION sub_name ENABLE` command to re-attach the subscription to the publication.

Making a logical replication using a physical replication instance

On PostgreSQL 16, it is possible to create a logical replication starting from a physical replication.

The steps that we have to take to do this are:

1. Set `wal_level=logical` on the **Primary** and **Physical Replication** server.

On the **Primary** server:

2. Create a role for the physical replication:

```
CREATE ROLE replicarole WITH REPLICATION LOGIN PASSWORD  
'LearnPostgreSQL'.
```

3. Create a role for the logical replication on the **Primary** server:

```
CREATE ROLE logicalreplicarole WITH REPLICATION LOGIN PASSWORD  
'LearnPostgreSQL'
```

4. Assign the correct permissions to the `logicalreplicarole` to the schemas and tables that we want to replicate by the logical replication, for example:

```
GRANT USAGE ON SCHEMA forum TO logicalreplicarole;  
GRANT SELECT ON forum.users TO logicalreplicarole
```

5. Create a physical replication slot or the physical replication.
6. Create a publication that will be replicated on the **Physical replication** server and used by the logical replication subscription on the **Logical Replication** server.

On the **Physical replication** server:

7. Set `hot_standby_feedback = on` to prevent problems due to vacuum operations on the primary server that are reflected on the replica server, which can create conflicts on very long queries on the replica.
8. Make the replica using a `pg_basebackup` command, as described in the previous chapter.

On the **Logical replication** server:

9. Create the tables that you want to replicate the data from physical replication.
10. Create a subscription that refers to the publication created on the primary as we've seen.

Now, let's try this feature using Docker containers; the scenario we want to try is:

11. A `forumdb` database on the primary server
12. A physical replica of the whole cluster on the replica server

13. A logical replication of the `forum.users` tables on the logical replication server.

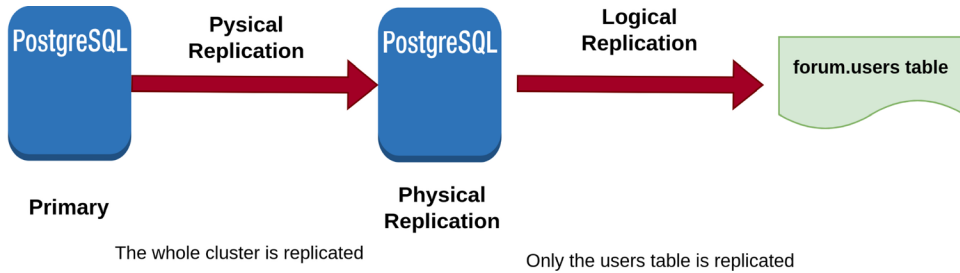


Figure 18.3: The physical/logical replication cascade

Let's use the `chapter18_physical_logical` Docker containers and execute the following steps:

1. Start all the containers using:

```
$ bash run-pg-docker-replica-logical.sh chapter18_physical_logical
```

2. After executing the statement above on our Docker host, we have three containers running:

```
chapter18_physical_logical_learn_postgresql_replica_sub_1
chapter18_physical_logical_learn_postgresql_master_pub_1
chapter18_physical_logical_learn_postgresql_replica_1
```

3. The script we ran took us directly inside the container where the primary node runs; now, we have to open two bash terminal windows and execute, on the first one, the statement below (to enter the physical replication node):

```
$ bash run-pg-docker-replica1.sh chapter18_physical_logical
```

4. On the second one, we have to execute the statement below to enter the logical replication node:

```
bash run-pg-docker-replica2.sh chapter18_physical_logical
```

5. Now, as we can see, on the primary node we have:

```
postgres@pg_master_pub:~$ psql

postgres=# select * from pg_stat_replication;
-[ RECORD 1 ]-----+-----
```

```

pid          | 108
usesysid     | 16384
username     | replicarole
application_name | walreceiver
client_addr  | 172.29.0.4
client_hostname |
client_port  | 43400
backend_start | 2023-06-20 16:10:30.779359+00
backend_xmin  |
state        | streaming
sent_lsn     | 0/3005948
write_lsn    | 0/3005948
flush_lsn    | 0/3005948
replay_lsn   | 0/3005948
write_lag    |
flush_lag    |
replay_lag   |
sync_priority | 0
sync_state   | async
reply_time   | 2023-06-20 16:11:49.332655+00

```

On the physical replication server, we have:

```

postgres@pg_replica:~$ psql forumdb

forumdb=# select * from pg_stat_replication;
-[ RECORD 1 ]-----+-----
pid          | 101
usesysid     | 16468
username     | logicalreplicarole
application_name | users_sub
client_addr  | 172.29.0.2
client_hostname |
client_port  | 43950
backend_start | 2023-06-20 16:10:39.685477+00
backend_xmin  |
state        | streaming
sent_lsn     | 0/3005948

```

```

write_lsn      | 0/3005948
flush_lsn      | 0/3005948
replay_lsn     | 0/3005948
write_lag      |
flush_lag      |
replay_lag     |
sync_priority  | 0
sync_state     | async
reply_time     | 2023-06-20 16:11:39.389851+00

```

6. So let's try to make some operations; on the primary server, try entering:

```

forumdb=# select * from forum.users;
 pk | username | gecos |          email
-----+-----+-----+-----
   1 | enrico   | 1     | enrico.pirozzi@packtpub.xyz
(1 row)

```

On the physical replication server, enter:

```

forumdb=# select * from forum.users;
 pk | username | gecos |          email
-----+-----+-----+-----
   1 | enrico   | 1     | enrico.pirozzi@packtpub.xyz
(1 row)

```

And on the logical replication serve, enter:

```

postgres@pg_replica_sub:~$ psql forumdb

forumdb=# select * from forum.users;
 pk | username | gecos |          email
-----+-----+-----+-----
   1 | enrico   | 1     | enrico.pirozzi@packtpub.xyz
(1 row)

```

7. Now, let's try to delete a record on the physical replication server:

```

forumdb=# delete from forum.users ;
ERROR:  cannot execute DELETE in a read-only transaction

```

8. And let's try to insert a record on the logical replication server:

```
forumdb=# insert into forum.users (pk,username,gecos,email) values
(2,'luca',1,'luca.ferrari@packtpub.xyz');
INSERT 0 1
```

```
forumdb=# select * from forum.users order by pk;
 pk | username | gecos |          email
----+-----+-----+-----
  1 | enrico   | 1     | enrico.pirozzi@packtpub.xyz
  2 | luca     | 1     | luca.ferrari@packtpub.xyz
(2 rows)
```

9. Now, let's try to delete a record on the primary server:

```
forumdb=# delete from forum.users where pk =1 ;
DELETE 1
```

10. And let's see what happened on the physical replication server:

```
forumdb=# select * from forum.users;
(0 rows)
```

11. And on the logical replication server:

```
forumdb=# select * from forum.users;
 pk | username | gecos |          email
----+-----+-----+-----
  2 | luca     | 1     | luca.ferrari@packtpub.xyz
(1 row)
```

Summary

In this chapter, we discussed logical replication. We saw that logical replication is based on a concept of reverse engineering, starting with the analysis of WAL segments to extract the logical commands that have to be passed to a replica server. We saw that logical replication is useful when we want to replicate parts of databases and when we want to make hot migrations between different versions of PostgreSQL. Logical replication makes this possible because it does not bi-narily replicate data but, rather, extracts the logical DML commands from WAL files, which are then replicated on the replica server.